IT E
Batch 1
G4

19 Punyesh Kapre SY-IT E
20 Narayani Rajebhosale SY-IT E
21 Karan Rajput SY-IT E
23 Chinmay Rane SY-IT E
24 Amber Rathi SY-IT E

Early-Stage Confidentiality Risk Detection Using Software Architecture Models

Introduction :

The software security attribute of confidentiality protects sensitive information by permitting access only to authorized system components. Design phase activities produce most confidentiality problems because this stage involves making decisions about component architectures and data movement patterns and user access permissions. The unstructured nature of architecture documents together with their use of informal creation methods makes it difficult to identify security risks through manual testing methods.

The traditional review process occurs during the latter part of development which makes it expensive to correct confidentiality breaches that become known after the system has been built. Architects currently lack a simple design-time tool which enables them to evaluate architectural designs for confidentiality risks during the design process.

This project solves the existing problem through a rule-based system which detects confidentiality risks by assessing architectural models before their implementation. The system evaluates components together with their data flow patterns and role access rights to find confidentiality risks which it resolves through secure design recommendations.

Purpose :

The document establishes the specifications which define the requirements of the Early-Stage Confidentiality Risk Detection System.

The system seeks to:

- The system needs to examine data flows and access roles and architectural elements.
- The system needs to identify all potential breaches which may violate confidentiality .
- The system needs to classify all threats based on their level of importance.
- The system needs to provide architectural mitigation techniques which function as its safeguard solutions .
- The system exists to support analytical work and educational activities by helping developers and architects and students learn secure software design methods.

Scope :

The system restricts its scope to identify confidentiality risks that occur during the design process through software architecture models.

The system requires architectural details as input which it processes according to its existing confidentiality rules to detect and classify security risks while creating recommendations for risk reduction.

The system does not conduct monitoring activities during system operation or security testing or automated compliance evaluation. The software functions as a decision-making assistance system rather than a system that enforces rules.

Definitions, Acronyms, and Abbreviations :

1. **Definitions**

- Software Architecture: The high-level structure of a software system shows its components together with their specific roles and the ways that they will interact with one another.
- Confidentiality: The security property of a system protects sensitive information by permitting access only to users and systems that have received proper authorization.
- Confidentiality Risk: A particular software architecture flaw exists which enables unauthorized parties to access confidential information.
- Design-Time Analysis: The evaluation of software artifacts takes place during the requirement and design phases before the software system enters its implementation stage.

- Data Flow: The movement of data occurs between software components that exist within a system architecture framework.
- Sensitive Data: Information needs protection because it contains personal data and confidential information which creates privacy and security issues and breaks regulatory rules.
- Access Control: The system uses access control mechanisms to limit resource access according to user roles and permissions.
- Risk Severity:The classification system defines three levels of impact for confidentiality risks which include Low and Medium and High.
- Mitigation Strategy: The document identifies architectural changes and control measures which should be implemented to decrease or remove all detected confidentiality risks.

2.**Acronyms**

Acronym	**Description**
SRS	Software Requirement Specification
SDLC	Software Development Life Cycle
UI	User Interface
RBAC	Role-Based Access Control
HTTPS	HyperText Transfer Protocol Secure

PII	Personally Identifiable Information

3.**Abbreviations**

Abbreviation	**Meaning**
Req.	Requirement
Arch.	Architecture
Comp.	Component
Sec.	Security
Auth.	Authorization
Viz.	Visualization

References:

[1] Schwickerath, F., Boltz, N., Hahner, S., Walter, M., Gerking, C., Heinrich, R. (2023).
Tool-Supported Architecture-Based Data Flow Analysis for Confidentiality. arXiv preprint arXiv:2308.01645.
[2] Orellana, C. (2025).
Architectural Tactics and Trade-offs for Confidentiality. Journal of Systems and Software, Elsevier.
[3] Chondamrongkul, N., et al. (2021).
Formal Security Analysis for Software Architecture Design. Science of Computer Programming, Elsevier.
[4] Hahner, S. (2021).
Dealing with Uncertainty in Architectural Confidentiality. Karlsruhe Institute of Technology.
[5] Hahner, S. (2024).
Architecture-Based and Uncertainty-Aware Confidentiality Analysis. PhD Dissertation, Karlsruhe Institute of Technology.
[6] Bourdelles, M. (2024).
Ensuring Requirements Confidentiality during System Design. ACM Digital Library.
[7] Dal Lago, L., et al. (2022).
Risk-Driven Model-Based Architecture Design for Secure Information Flows. Proceedings of SECRYPT 2022.
[8] Sabau, A. R., et al. (2025).
SecuRe — An Approach to Recommending Security Design Patterns. arXiv preprint.
[9] Ding, Y., Sato, H. (2024).
Model-Driven Security Analysis of Self-Sovereign Identity Systems. arXiv preprint.
[10] Abdulaziz, F. J., Jürjens, J. (2020).
Modeling and Analysis of Secure Systems using UML and OWL. Journal of Information Security and Applications, Elsevier.
[11] Scandariato, R., et al. (2014).
Security in Software Engineering: A Survey. IEEE Transactions on Software Engineering.
[12] Verdecchia, R., et al. (2018).
Architectural Smells and Their Impact on Software Quality. Empirical Software Engineering.
[13] Gotel, O., Cleland-Huang, J. (2012).
Requirements Traceability. Springer.
[14] Spanoudakis, G., Zisman, A. (2005).
Software Traceability: A Roadmap. Proceedings of ICSE.
[15] Usman, M., Imran, A. (2021).
A Systematic Approach to Secure Software Architecture Design. IEEE Access.
[16] Gonzalez, L., et al. (2018).
Architectural Patterns for Confidential Data Protection. Journal of Systems and Software, Elsevier.
[17] Mahajan, R., et al. (2019).
Data Flow Integrity and Confidentiality in Component-Based Systems. IEEE Secure Software Engineering.
[18] Mavridou, A., Pasquale, L. (2021).
Embedding Security in UML Models using OCL Constraints. ACM/IEEE MODELS Conference.

Product Perspective:

The proposed system operates as a standalone web-based analysis tool which functions during the requirement and design phase of the software development life cycle. The system enhances existing security tools by providing additional detection capabilities which identify confidentiality risks at their earliest stages.

The system follows a modular pipeline which consists of these components:

- Architecture input
- Data flow modeling
- Rule-based risk analysis
- Risk classification
- Mitigation recommendation

Product Functions:

- The software architecture details must be accepted by you.
- The system requires modeling for its components and data flows and the definition of access rights.
- The system needs to identify all data flows which contain sensitive information.
- The system requires the implementation of rules which validate confidentiality requirements.
- The system will identify security threats and categorize them into different danger levels.
- The system will create security risk reduction recommendations.
- The system uses a dashboard to present its output results.

User Classes and Characteristics:

1. **Software Architect**
- Creates system architecture design
- The tool assesses confidentiality risks throughout the entire project lifecycle.
2. **Software Developer**
- Develops security knowledge to evaluate design risks.
- The team reviews their proposed solutions for risk reduction.
3. **Project Manager**
- He monitors all design risks while executing his project responsibilities.

- He uses the reports to support his project planning activities.

4. **Students / Researchers**

- They study secure design methods.
- They utilize the system for their academic research work.

Operating Environment:

1. **Client Side**

- Web browser (Chrome, Firefox, Edge)
- OS: Windows / Linux / macOS

2. **Server Side**

- Backend server (Python / Node.js)
- Rule-based analysis engine

3. **Security**

- HTTPS communication
- No permanent storage of sensitive input

Design and Implementation Constraints:

The Early-Stage Confidentiality Risk Detection System requires these specific limitations which help develop its design and functions while maintaining transparent operations and relevant research goals.

1.Design-Time Analysis Constraint

The system restricts its software architecture model analysis capabilities to the requirement and design phases of software development. The system does not support execution monitoring or analysis of active systems through deployment or runtime operations.

2.Rule-Based Analysis Constraint

The system uses established security rules to detect confidentiality breaches. The system does not feature machine learning elements or any adaptive risk assessment methods.

3.Security Attribute Constraint

The system operates solely to develop the confidentiality quality attribute. The project does not include development of any attributes except for the confidentiality quality attribute.

4.Architecture Abstraction Constraint

The system functions through abstract architectural models that include components and data flows and access roles instead of using actual source code and infrastructure details.

5.Limited System Scale Constraint

The tool serves educational and academic projects through its capability to handle small to medium-scale software architectures.

6.Technology Constraint

The system operates through standard web technologies while using open-source software components. The system does not utilize proprietary security analysis platforms or licensed security tools.

7.User Input Dependency Constraint

The system needs correct and complete architecture data from users to achieve accurate risk detection results.

8.Time and Academic Constraint

The system design and implementation process needs to follow both semester time limits and academic assessment standards.

User Documentation:

- The user accesses the system through a web interface using a standard browser.
- The user defines software architecture components, data stores, and access roles.
- The system requires users to provide data flows together with their associated data sensitivity and protection requirements.
- The system analyzes the architecture using predefined confidentiality rules.
- The system displays identified confidentiality risks which have been categorized by their level of severity.
- The system provides architectural mitigation suggestions to reduce risks.

Assumption and Dependencies:

- The architectural models provide accurate representation of their intended system design.
- Users provide correct data classification (sensitive/non-sensitive)
- The system defines confidentiality rules which remain unchanged throughout its operation.
- The system functions exclusively during the design phase of the project.
- Users can access the web because they have internet connectivity.

System Features:

- Architecture component definition
- Data flow and data sensitivity modeling
- Confidentiality rule evaluation
- Risk severity classification
- Mitigation strategy recommendation
- Risk visualization dashboard

Function Requirements:

FR1: Users should have the ability to create their own architectural components.

FR2: The system should enable users to create models of data movement through their components.

FR3: The system should enable users to determine the protection level of their data through system classification.

FR4: The system should implement confidential information protection standards through its validation procedures.

FR5: The system should detect all potential threats to confidential information security.

FR6: The system should categorize risks into three levels which are Low, Medium and High.

FR7: The system should provide users with recommended methods to reduce identified risks.

FR8: The system should show users the results of its risk assessment process.

External Interface Requirements:

The proposed system establishes its user interfaces through which users and system elements interact with the system**.**

User Interface

•Architecture Input Interface:

The system enables users to create software components through its structured interface which defines data flows and access roles and data sensitivity attributes.

•Risk Visualization Dashboard:

The system presents discovered confidentiality risks through basic visual tools which show their danger levels.

•Mitigation Report View:

The system displays all detected risks together with their respective mitigation strategies in a complete report that users can download.

Software Interface

•Rule-Engine Module:

The system uses established confidentiality rules to evaluate architectural inputs in order to identify possible security breaches.

•Data Analysis Module:

The system examines data flows and component interactions to assist in discovering and categorizing risks.

User Interface:

The system provides a web-based user interface designed to be simple, intuitive, and suitable for both technical and non-technical users. The interface supports architecture modeling work, confidentiality risk assessment, and result display.

1. General Characteristics

- The user interface is accessible through standard web browsers.
- The system uses a dashboard-based design which enables users to navigate the interface through its organized layout.
- All screens use the same color scheme and label system and icon collection for their design.
- The system enables users to interact with it because its design requires less manual work while reducing the chance of making mistakes.

2. Major Interface Screens

2.1 Architecture Modeling Screen

- The system enables users to create software components and data stores and access roles through its defining function.
- The system enables users to create and modify and remove architecture elements through its system.
- The system provides users with simple form-based inputs.

2.2 Data Flow Definition Screen

- The system enables users to define their data source and destination component and data type and data protection mechanism requirements.
- The system requires users to validate their inputs before they can submit them.

2.3 Confidentiality Risk Analysis Screen

- The system begins its analysis through rule-based methods which assess the entire architectural structure.
- The system shows all detected confidentiality risks through its display.
- The system displays three risk severity levels which include Low and Medium and High.

2.4 Risk Visualization Dashboard

- The system displays results through different presentation formats which use both tables and charts.

- The system identifies components and data flows which have been classified as high risk.
- The system enables users to filter content based on different risk levels.

2.5 Mitigation Recommendation Screen

- The screen presents architectural mitigation strategies which the system recommends.
- The screen provides an explanation for every recommendation that follows the specific recommendation.
- The system enables users to create mitigation reports which they can export in various formats.

3. Error Handling and Feedback

- The system shows users descriptive error messages which explain their invalid input.
- The system provides users with confirmation messages that show successful completion of their actions.
- The system stops users from submitting data that contains incomplete information or contradictory details.

4. Usability Requirements

- First-time users should need only a short time to learn how to operate the system.

Hardware Requirements:

- Desktop or laptop computer
- Minimum 4 GB RAM
- Standard keyboard and mouse
- Internet-enabled device
- No specialized hardware required

Software Interface:

- Web browser (Chrome, Firefox, Edge)
- Backend application server
- Rule-based analysis engine
- Database system (MySQL / MongoDB)
- Visualization library support

Communication Interface:

- The client-server communication uses the HTTPS protocol.
- The REST-based API allows users to perform system interactions.
- The system protects data transmission through secure transmission methods.
- The system uses a stateless design to handle request and response operations.
- The system operates without needing any external systems for its communication.

Non-Functional Requirements:

- The system provides usable functions which users can easily learn to operate.
- The system delivers rapid analysis results with its fast response performance.
- The system supports architectural designs which require midrange capacity.
- The system maintains its functions through its design which uses separate components for processing rules.
- The system can operate on any computing platform without restrictions.

Performance Requirements:

- The architectural evaluation process finished within a few seconds.
- The system supports concurrent access by multiple users.
- The system enables effective management of architectural information.
- The system operates with minimal resource requirements.
- The dashboard displays information with quick loading speeds

Security Requirements:

- All frontend-backend communication requires the system to use HTTPS because this protocol ensures protection for data that travels between these two system components.

- The system must handle sensitive architecture inputs which include data sensitivity labels through secure methods that prohibit their storage in raw format.
- The system should restrict access to architecture models and analysis results which should only be accessible by authorized personnel.
- The backend system requires validation of all incoming requests to stop attackers from introducing harmful data into the system.
- The system needs to protect confidentiality rules together with internal configuration files from unauthorized access.
- The system needs to keep sensitive information out of logs because this practice protects against unintentional disclosure during the debugging process.

Reliability and Availability:

- The system shows continuous operational time during its active usage period.
- The system demonstrates effective performance by managing invalid inputs.
- The system provides users with precise information about errors that occurred during operation.
- The system produces identical outcomes for risk assessment through its consistent analytical process.
- Users can depend on consistent access to their stored architectural documents.

Maintainability:

The system uses a modular design framework which enables simple maintenance processes and supports future system extensions. The system consists of four independent modules which include architecture modeling, rule engine, risk analysis, and visualization. The system stores confidentiality rules in a flexible format which permits users to modify existing rules or create new ones. The system documentation together with its coding standards and design principles enables developers to learn system operations and system updates and system expansion. The system uses logging systems together with error management systems to assist developers in their debugging tasks and their system maintenance activities.

Portability:

The system can operate on any system which has a modern web browser installed because it does not depend on specific platforms. The backend system uses standard web technologies which allow its operation on Windows and Linux and macOS without any need for changes. The system allows cloud hosting and server migration because its rule engine and analysis logic operate from server locations. The system achieves easy movement between various operating systems because it requires only basic installation and uses common frameworks which have extensive support.

Data Requirements:

Database Design Constraints:

- The database needs to maintain a structured and consistent format for storing components together with data flows and roles and risk outputs.
- The database requires all relationships to maintain referential integrity which includes data flow and component relationships.
- The system needs to secure sensitive input data while preserving all confidential information through restricted access.
- The schema should enable users to create or modify confidentiality rules through simple procedures.
- The database needs to provide quick access to data which supports both risk analysis and report creation.
- The selected database management system must work across different platforms while also supporting web-based applications.

Validation and Verification:

Verification

The process of verification checks whether the system development meets the established requirements. The team verifies all functional and non-functional requirements through requirement reviews and rule validation and consistency checks to ensure accurate implementation of confidentiality analysis logic.

Validation:

Validation ensures that the developed system meets user needs and intended objectives. The system validation process evaluates whether the identified confidentiality risks and mitigation suggestions provide meaningful and accurate and useful information during the software design phase.

Future Enhancements:

- The system needs the ability to protect three security elements which include integrity and availability and authorization together with its existing confidentiality protection.
- The system requires implementation of compliance verification checks which assess compliance with both GDPR and ISO/IEC 27001 standards.
- The system needs to implement machine-learning analysis which will use historical architectural data to forecast risk patterns.
- The system requires functionality to import and export architectural diagrams through tools which include Draw.io and Lucidchart.
- The system needs to display additional risk information through heatmaps and multiple architectural presentation methods.
- The system enables teams to work together through features which include commenting, tracking changes, and conducting joint evaluations.

Appendices:

Appendix A: Glossary

- The glossary provides definitions for important terms which relate to security and architecture.
- The document provides an explanation of confidentiality-related concepts.
- The document defines three levels of risk severity.
- The document contains standard abbreviations which are used throughout.
- The document defines terms which describe the architectural structure and data movement in the system.

Appendix B: Architecture Diagrams

- The diagram shows the complete system architecture at a high level.
- The diagram shows how system components interact with each other.
- The diagram shows how data flows through the system.
- The workflow establishes the procedures for conducting confidentiality analysis.

- The document includes a visual reference which shows the different levels of risk.

Appendix C: Use Case Details

- The document provides complete descriptions of all use cases.
- The document defines all actors and their respective duties within the system.
- The document describes required conditions for the system to start and end its operations.
- The document shows the standard operating procedure together with its backup procedures.
- The document identifies all assumptions that apply to use cases.

Appendix D: Rule Definitions

- The document presents all rules which protect confidentiality within the system.
- The document explains the procedures which assess rules.
- The document establishes criteria for classifying various risks.
- The document describes the rules which determine appropriate mapping for risk mitigation.
- The document describes the extent to which the rules can be applied.

Appendix E: Sample Inputs and Outputs

- The document provides an architecture example which shows the system's input requirements.
- The document provides example data flow definitions which need to be used.
- The document provides example results for risk analysis procedures.
- The document provides example risk reduction strategies which can be used.
- The document includes screenshots which show the output results from the dashboard.

Appendix F: Revision History

- The document shows the version number which identifies the current document.
- The document shows the date when the document received its latest update.

- The document provides a summary of all modifications which were made to the document.
- The document provides the name of the person who made the document modifications.
- The document shows whether the document received approval or not.